

Tool 实现能力清单（含 Mock 行为 + 工程证据）

本文件用于满足赛题交付物要求：说明系统在“把事情做完”链路中已落地的工具能力，并提供可验收的工程证据。

说明：为便于评委快速验收，本文件正文不写绝对路径；仅提供**相对路径 + 函数名索引**，并贴出少量关键实现片段（节选），用于证明链路真实可复现。

Tool 总览（按用户动作闭环）			
用户动作	对应工具/机制	目的	常见失败模式（摘要）
一句话生成方案（首次/再规划）	plan_trip	将结构化约束发送至规划子服务并落盘到 state（方案 + 候选池）	子服务不可用/超时（≤3s）
想多看备选	generate_alternative_plans	在不改约束前提下扩展备选列表	子服务不可用/超时
先搜再换（模拟推荐/搜索）	search_candidates	在当前候选池内按用户关键词做二次检索	搜索服务超时/无结果
指定替换某一条目	adjust_plan_item	替换条目并触发确定性冲突修复（2 层最小改动：吃 buffer → 压缩；不允许删项）	替换导致超窗/超预算、无可修复
执行失败后补齐并继续	adjust_and_execute_plan_item + fixup_agent 路由	fixup 阶段一次性完成“替换 + 重试执行”	备选不安全/仍失败
确认后执行（Mock）	execute_itinerary + mock_executor + /execution/{id}/commit	先生成待支付执行命令，输入 111111 后 Mock commit	无座/无票/库存不足、支付失败、执行超时、一致性校验失败

1) 规划/再规划：plan_trip（同一入口）

1.1 功能边界

- 输入：结构化约束（群体 family/friends、预算、开始时间、4–6h 窗口、距离偏好、活动偏好、忌口、排队偏好等）。
- 输出：结构化方案（默认 1 个最优方案）+ 候选池（用于后续搜索/替换）。
- 性能约束：工具预算默认 ≤3s；超时会返回可恢复错误，引导用户重试或调整输入（保证端到端稳定）。

1.2 工程证据点（评委验收关注）

- **内部 HTTP 调用规划子服务（8001）**：多地址候选 + 总超时 budget 控制。
- **状态落盘**：constraints / candidates / itinerary / latest_plan_result 写回 state，后续 search/replace 都基于该 state。
- **可追溯日志（ELK）**：已预留 ELK 接入与结构化 JSON 日志输出能力；结合 phase / session_id / tool / error 等字段，可以在 Kibana 或离线日志中按链路追溯规划过程，便于快速排查超时、候选不足、剪枝过多等问题。

1.3 方案生成处理顺序（非黑盒）

- **主图入口职责 (plan_trip)**：先做结构化约束归一化与工具 budget 控制，再调用 8001 规划子服务；成功后把 constraints / candidates / itinerary / latest_plan_result 回写到主状态。
- **子图固定顺序**：规划子服务不是单次 prompt 出方案，而是按固定节点顺序执行：retrieve_candidates_node -> filter_node -> rerank_node -> planner_node。
- **粗召回 (retrieve)**：从 Mock DB 拉取餐厅、活动、礼品三类候选，并补齐/校正距离字段，先得到可消费的候选池。
- **硬过滤 (filter)**：按距离、人群适配、饮食禁忌、人数与年龄、礼品配送半径等硬约束剔除不满足要求的候选，避免无效组合进入后续阶段。
- **重排 (rerank)**：按场景契合、热度质量、排队偏好、活动时长倾向等信号重新打分排序，把更像“当前用户会选”的候选放到前面。
- **Pattern 排列组合**：planner_node 会先匹配预定义 pattern；如果用户给了 preferred_pattern_steps，会优先做子序列匹配，必要时构造自定义 pattern（含 gift 插入）。
- **DFS 枚举 + 剪枝**：在每个 pattern 对应的候选池上做深度组合，同时对预算、总时长、步行距离、返程距离、饭点合法性、礼品配送范围、超过 4 小时必须有餐饮等条件做剪枝。
- **最终排序出 plan**：只对通过全部约束的合法组合计算综合分，再做去重与差异化筛选，最后返回 Top-K 中最优的 plan 结果。
- **备选与确认边界**：plan 产出后，系统还会为步骤预埋 backup candidates，并继续计算“在尽量不破坏当前整体方案的情况下是否安全可替换”。如果替换可能触碰硬约束（预算/时长）或损伤软偏好（适配标签），则会标记 requires_confirmation=true，后续执行/fixup 阶段必须让用户确认，而不是静默替换。

1.4 失败模式

- **规划子服务超时/不可用**：返回 TOOL_TIMEOUT（可恢复），不阻塞后续对话。
- **上游异常**：返回 UPSTREAM_ERROR（可恢复），并回传当前约束用于用户修改。

1.5 验收动作 (How to Verify)

- 关闭规划子服务或设置极短超时，触发 TOOL_TIMEOUT，确认工具返回可恢复错误且不崩溃。
- 恢复子服务后再次调用，能正常产出方案并写回 itinerary/latest_plan_result。
- 观察子图日志或代码实现，确认真实链路存在 粗召回 -> 硬过滤 -> 重排 -> pattern/DFS -> 排序出 plan 的固定顺序，而不是单次 prompt 直接吐 itinerary。
- 在执行阶段人为制造一个“更贵/更远/适配性更差”的备选，确认其会被标记为需要用户确认，而不是直接静默替换。

1.6 实现索引与代码片段 (节选)

- 实现索引：
- backend/src/closedloop/graph/tools/plan_tool.py::plan_trip
- backend/src/closedloop/graph/tools/plan_sub_api.py::request_plan_sub_json
- backend/src/closedloop/graph/plan_subgraph/builder.py::build_subgraph_plan
- backend/src/closedloop/graph/plan_subgraph/retrieve.py::retrieve_candidates_node / filter_node
- backend/src/closedloop/graph/plan_subgraph/rerank.py::rerank_node
- backend/src/closedloop/graph/plan_subgraph/planner.py::planner_node
- backend/src/closedloop/graph/plan_subgraph/planner_utils.py::generate_and_score_combinations

片段 1：工具预算 + 内部调用 + 可恢复失败码 (节选)

```
# backend/src/closedloop/graph/tools/plan_tool.py::plan_trip (节选)
tool_budget_secs = float(getattr(config, "TOOL_MAX_RUNTIME_SECS", 3.0))
started_at = time.perf_counter()

remaining_secs = tool_budget_secs - (time.perf_counter() - started_at)
if remaining_secs <= 0:
```

```

        raise TimeoutError("tool_budget_exhausted")

subgraph_output = request_plan_sub_json(
    method="POST",
    configured_url=getattr(config, "PLAN_SUB_API_URL", "http://localhost:8001/plan"),
    target_path="/plan",
    phase="plan_trip",
    json=payload,
    timeout=remaining_secs,
    network_mode=getattr(config, "PLAN_SUB_NETWORK_MODE", "local"),
)

```

片段 2: 多地址候选 + 总超时 budget (节选)

```

# backend/src/closedloop/graph/tools/plan_sub_api.py::request_plan_sub_json (节选)
candidate_urls = build_plan_sub_candidate_urls(configured_url, target_path, network_mode=network_mode)
deadline = started_at + float(timeout)

for attempt_index, candidate_url in enumerate(candidate_urls, start=1):
    remaining_secs = deadline - time.perf_counter()
    if remaining_secs <= 0:
        raise TimeoutError("plan_sub_api_total_timeout")
    response = client.request(
        request_method,
        candidate_url,
        json=json,
        params=params,
        timeout=remaining_secs,
    )

```

片段 3: 规划子图固定顺序 (节选)

```

# backend/src/closedloop/graph/plan_subgraph/builder.py::build_subgraph_plan (节选)
workflow.add_node("retrieve_candidates_node", retrieve_candidates_node)
workflow.add_node("filter_node", filter_node)
workflow.add_node("rerank_node", rerank_node)
workflow.add_node("planner_node", planner_node)

workflow.add_edge(START, "retrieve_candidates_node")
workflow.add_edge("retrieve_candidates_node", "filter_node")
workflow.add_edge("filter_node", "rerank_node")
workflow.add_edge("rerank_node", "planner_node")
workflow.add_edge("planner_node", END)

```

片段 4: 备选若可能破坏约束/偏好则要求确认 (节选)

```

# backend/src/closedloop/graph/plan_subgraph/planner.py::planner_node (节选)
hard_cost_exceeded = (budget < float('inf')) and (new_total_cost > budget)
hard_time_exceeded = new_total_duration > max_duration
soft_violated = not old_suitable.issubset(new_suitable)

```

```
requires_confirmation = False
if hard_cost_exceeded or hard_time_exceeded or soft_violated:
    requires_confirmation = True

backup_candidates.append({
    "id": cand_id,
    "name": cand_name,
    "requires_confirmation": requires_confirmation,
    "violation_reason": "超出预算或时间" if (hard_cost_exceeded or hard_time_exceeded) else "偏好可能受损"
})
```

2) 候选搜索: search_candidates (模拟推荐/搜索)

2.1 功能边界

- 输入: 类目 (restaurant/activity/gift_shop) + 用户关键词 (user_request) + top_k。
- 输出: 基于当前候选池的搜索结果列表 (并返回命中原因, 用于解释)。
- 设计取舍: 优先保证响应时间与稳定性; 搜索子服务不可用时, 工具返回可恢复错误, 引导用户“换词/放宽/改用再规划”。

2.2 失败模式

- **搜索服务超时**: 返回 TOOL_TIMEOUT (可恢复)。
- **无结果**: 返回“没有找到结果”, 提示换词/放宽/重新规划。

2.3 验收动作 (How to Verify)

- 输入一个明显不相关的关键词, 确认前端 Agent 会明确回复“没有搜到”或等价提示, 并且当前方案不被破坏。
- 停止搜索服务 (若启用) 或制造超时, 确认前端 Agent 会明确回复“搜索超时/稍后再试/建议重新规划”等可恢复提示, 而不是卡死。
- 输入一个正常关键词, 确认前端 Agent 会展示搜到的候选结果, 并给出后续可替换入口。

2.4 实现索引与代码片段 (节选)

- 实现索引: backend/src/closedloop/graph/tools/search_tool.py::search_candidates

```
# backend/src/closedloop/graph/tools/search_tool.py::search_candidates (节选)
tool_http_timeout_secs = float(getattr(config_app, "TOOL_HTTP_TIMEOUT_SECS", 3.0))
configured_url = getattr(config_app, "SEARCH_SUB_API_URL", "http://127.0.0.1:8002/search")
payload = {"category": category, "user_request": user_request, "top_k": top_k, "candidates": ranked_cand

try:
    with httpx.Client(timeout=tool_http_timeout_secs, trust_env=False, proxy=None) as client:
        response = client.post(configured_url, json=payload)
        response.raise_for_status()
        res_json = response.json()
except httpx.TimeoutException:
    return Command(update={"messages": [fail_msg]})
```

3) 替换与冲突修复: adjust_plan_item (确定性修复: 2 层最小改动)

3.1 功能边界

- 输入：plan_id、target_item_id、new_item_id（通常来自 search_candidates 的结果）。
- 输出：替换后的新方案（尽量保持整体不变），并更新 itinerary/latest_plan_result。
- 冲突修复：确定性 2 层最小改动（只改时长与通勤重算，不改“项目集合”，更不会删项）。
- L1 吃 buffer：对非锁定条目，最多吃掉该条目的 duration_std_dev（将超出的分钟数优先从浮动里拿回来）。
- L2 压缩可压缩条目：仅对 activity/gift_shop 的非锁定条目做压缩，最低压到原始 duration_mins 的 60%（正餐不压缩）。
- 通过条件（与实现一致）：total_cost <= budget * 1.2 且 total_duration_minutes <= duration_max + 45min 宽容度。
- 说明：更“近/更短/更便宜”的邻近项替换（L3）为预留策略，当前实现未启用；一旦需要删项或仍无法修复，工具直接失败交由 Agent 协商（避免静默破坏体验）。

3.2 失败模式

- **替换导致严重超窗/超预算**：当自动修复需要“删项/破坏体验”或无法确定性修复时，工具返回失败消息；由 Plan Agent 基于失败原因引导用户“换备选/放弃替换/改约束再规划”。
- **无可替换候选/找不到 plan_id**：返回错误并提示重新搜索或再规划。

3.3 验收动作 (How to Verify)

- 对某个条目替换为明显更贵/更远的选项，触发“不能静默执行”的失败提示，确认 Agent 会引导用户“换备选/放弃替换/重新规划”，而不是强行删项。
- 对某个条目替换为一个正常推荐且可行的候选，确认当前 plan 会被直接替换，价格、时间与条目明细也会同步更新。

3.4 实现索引与代码片段（节选）

- 实现索引：
- backend/src/closedloop/graph/tools/adjust_tool.py::adjust_plan_item
- backend/src/closedloop/graph/plan_subgraph/repairer.py::repair_plan

片段 1：禁止静默“破坏性修复”（节选）

```
# backend/src/closedloop/graph/tools/adjust_tool.py::_do_adjust_plan_item (节选)
if status == "success" and "plan" in result:
    original_steps_count = len([s for s in target_plan.get("steps", []) if s.get("item", {}).get("type")
    new_steps_count = len([s for s in result["plan"].get("steps", []) if s.get("item", {}).get("type") !=
    if new_steps_count < original_steps_count:
        return "failed", {}, "替换该备选会导致总时间或总预算严重超标，系统尝试删除了您的其他活动...请您选择其他
```

片段 2：修复器入口 + “需要用户选择”的结构化输出（节选）

```
# backend/src/closedloop/graph/plan_subgraph/repairer.py::repair_plan (节选)
def repair_plan(plan: dict, target_item_id: str, new_item: dict, budget: float, duration_range_mins: tuple)
    logger.info(f"phase=repairer | plan_id={plan.get('plan_id')} | target={target_item_id} | new_item={n
    ...
    if target_index == -1:
        return {"status": "need_user_choice", "report": {"reason": f"在方案中找不到要替换的条目: {target_i
```

4) 执行 (Mock) : execute_itinerary + mock_executor (付款门控后 commit)

4.1 功能边界

- 输入: `plan_id + 通勤预约策略 (first_only/all)` 。
- 输出: 待支付执行命令 (`confirmation.status=pending_payment`) 或 `needs_fixup/timeout/failed`。
- Mock 行为: 不会真实下单; `execute_itinerary` 只做可用性检查与命令生成, 输入 Mock 支付密码 111111 后调用 `/execution/{id}/commit` 才写入运行时数据。

4.2 Mock 行为 (Side Effects, 可验收)

- **付款前**: 只生成待支付命令, 不扣减 `stock / available_stock / capacity_remaining` 。
- **付款失败**: 密码不是 111111 时返回 `payment_status=failed`, 不进入 `commit`。
- **付款成功**: 调用 `/execution/{id}/commit` 且密码正确后返回 `payment_status=paid`; 在 `commit` 阶段才会写入运行时数据 (礼品扣 `stock`, 活动扣 `available_stock`, 餐厅/活动预约扣对应时段 `capacity_remaining`) 。
- **失败兜底**: 仅对“允许自动替换”的餐厅条目尝试备选替换; 如备选需要用户确认则进入 `needs_fixup` 。

4.3 稳定性证据: 付款门控 + 一致性校验

- 付款前无运行时写入; 一致性校验用于守住执行命令与付款金额, `commit` 阶段才产生 Mock 副作用, 保证演示可复现、状态不污染。

4.4 验收动作 (How to Verify)

- 执行一次方案后, 在前端看到 `pending_payment` 与执行命令; 同时查看 `backend` 日志, 确认此时还没有实际扣减库存/容量。
- 输入错误密码, 确认前端返回支付失败; 同时查看 `backend` 日志, 确认没有进入真正的 `commit` 写入。
- 输入 111111, 确认 `commit` 后 `backend` 日志中能看到库存/容量写入或 `before/after` 变化信息。
- 人为制造无座 (`capacity=0` 或删除时段记录), 查看 `backend` 日志并结合前端表现, 确认系统进入备选替换或 `needs_fixup` 。

4.5 实现索引与代码片段 (节选)

- 实现索引:
- `backend/src/closedloop/graph/tools/execute_tool.py::_snapshot_runtime_jsons / _restore_runtime_jsons`
- `backend/src/closedloop/graph/tools/execute_tool.py::execute_itinerary / _do_execute_itinerary`
- `backend/src/closedloop/execution/mock_executor.py`

片段 1: 运行时快照/回滚 (节选)

```
# backend/src/closedloop/graph/tools/execute_tool.py (节选)
def _snapshot_runtime_jsons(rw_dir: str, filenames: list[str]) -> dict[str, Any]:
    snapshot: dict[str, Any] = {}
    for name in filenames:
        path = os.path.join(rw_dir, name)
        if not os.path.exists(path):
            continue
        with open(path, "r", encoding="utf-8") as f:
            snapshot[name] = json.load(f)
    return snapshot

def _restore_runtime_jsons(rw_dir: str, snapshot: dict[str, Any]) -> None:
    for name, content in (snapshot or {}).items():
        path = os.path.join(rw_dir, name)
        tmp_path = f"{path}.tmp"
        with open(tmp_path, "w", encoding="utf-8") as f:
            json.dump(content, f, ensure_ascii=False, indent=2)
        os.replace(tmp_path, path)
```


片段 2：一致性校验失败回滚与错误码（节选）

```
# backend/src/closedloop/graph/tools/execute_tool.py::_do_execute_itinerary（节选）
if overpay:
    _restore_runtime_jsons(rw_dir, runtime_snapshot)
    message = "执行一致性校验失败（预算超标或行程不匹配）：已回滚本次执行，请重新调整您的行程方案。"
    result = {"code": "EXECUTION_INCONSISTENT_NEEDS_RETRY", "message": message}
    confirmation = {"status": "failed", "code": "EXECUTION_INCONSISTENT_NEEDS_RETRY", "message": message}
```

片段 3：Mock 扣库存/扣容量（节选）

```
# backend/src/closedloop/execution/mock_executor.py（节选）
elif gift and isinstance(gift.get("stock"), int):
    before_stock = int(gift["stock"])
    if before_stock > 0:
        gift["stock"] = before_stock - 1
        reserved = True
        _atomic_write_json(os.path.join(repo_dir, "add_ons.json"), add_ons)

elif requires_booking:
    ok, detail = _reserve_capacity(reservations, "package", step.item_id, step.start_time)
    if ok and isinstance(pkg.get("available_stock"), int) and int(pkg["available_stock"]) > 0:
        pkg["available_stock"] = int(pkg["available_stock"]) - 1
        _atomic_write_json(os.path.join(repo_dir, "reservations.json"), reservations)
```

5) 执行失败后的补齐（fixup_agent）：路由与工具边界

5.1 功能边界

- 触发条件：execute_itinerary 返回 confirmation.status == needs_fixup。
- fixup_agent 的职责：向用户展示候选1/2，或直接引导用户继续搜索；若用户输入关键词，则调用 search_candidates；若用户选中候选，则调用 adjust_and_execute_plan_item 完成“替换 + 重试执行”。
- 取舍说明（口径A）：采用“轻量协同”保证闭环成功率与响应稳定；只在关键失败点请求用户明确选择，其余尽量自动兜底。

5.2 验收动作（How to Verify）

- 人为制造一次 needs_fixup 场景，确认 Agent 会先展示候选1/2，或者提示用户可以继续搜索别的备选。
- 在 fixup 对话里直接输入搜索关键词，确认 Agent 会调用 search_candidates，并返回“没搜到 / 超时 / 搜到若干候选”中的一种可见结果。
- 在 fixup 对话里直接选择候选1/2，或从搜索结果中指定一个候选，确认 Agent 会调用 adjust_and_execute_plan_item 完成“替换 + 重试执行”。

5.3 实现索引与代码片段（节选）

- 实现索引：backend/src/closedloop/graph/agent.py::resolve_active_agent / apply_step_config

```
# backend/src/closedloop/graph/agent.py（节选）
def resolve_active_agent(state: dict) -> str:
    confirmation = state.get("confirmation", {})
    if isinstance(confirmation, dict) and confirmation.get("status") == "needs_fixup":
        return "fixup_agent"
    return state.get("active_agent", "plan_agent")
```

```
configs = {  
    "fixup_agent": {  
        "tools": [search_candidates, adjust_and_execute_plan_item]  
    }  
}
```

6) 数据依赖 (Mock DB)

- 静态数据（规划期读取）：餐厅/活动/礼物/预约数据（用于召回、过滤、排序与备选生成）。
- 运行时数据（执行期写入）：库存与预约容量扣减写入运行时目录，用于稳定复现“执行副作用”与“失败兜底”。